

PROYECTO INTEGRADO – 2.º DAM

Gestor de películas y series personalizado

Alumno: Jose Ángel Soriano Erenas

Tutor: Carlos Barroso Moriana

Ciclo: Desarrollo de Aplicaciones Multiplataforma (2.º DAM)

Curso: 2025 – 2026

Índice

1. Especificación del proyecto.....	3
2. Elementos no vistos en clase.....	4
2.1. PHP 8.2.....	4
2.2. CodeIgniter 4.....	4
3. Información sobre el proyecto.....	5
3.1. Arquitectura general.....	7
3.2. Diseño de la base de datos.....	8
3.3. Capturas de pantalla.....	13
4. Parte práctica.....	17
4.1. Herramientas utilizadas.....	17
4.2. Instalación del entorno de desarrollo.....	18
4.3. Despliegue en producción (Railway).....	19
5. Bibliografía.....	20

1. Especificación del proyecto

El proyecto consiste en el desarrollo de una aplicación móvil completa para la gestión personalizada de películas y series. El usuario puede buscar contenido a través de la API de TMDB (The Movie Database), organizarlo en listas propias, registrar puntuaciones y reseñas, y hacer seguimiento de los episodios de cada serie.

Especificaciones técnicas

Componente	Tecnología utilizada	Especificación original
Frontend	Flutter (Android)	Aplicación móvil con Flutter
Backend	PHP 8.2 + CodeIgniter 4	PHP + Laravel
Base de datos	PostgreSQL 16	PostgreSQL
API externa	TMDB API v3	Uso de APIs externas
Despliegue	Railway (PaaS)	—
Entorno de desarrollo	Docker + Docker Compose	—

Nota sobre el framework backend: la especificación original indicaba el uso de Laravel. Se decidió utilizar CodeIgniter 4 por ser un framework PHP de la misma familia (mismo ecosistema, mismo gestor de dependencias Composer, misma arquitectura Modelo-Vista-Controlador), más ligero y que ofrece mayor control sobre la estructura de la API REST. Las funcionalidades implementadas son equivalentes a las que se habrían desarrollado con Laravel.

Funcionalidades implementadas

Registro e inicio de sesión con autenticación JWT (access + refresh token).

Búsqueda de películas, series y personas a través de la API de TMDB.

Sistema de listas personalizadas con dos listas creadas automáticamente al registrarse.

Puntuaciones (0–10) y reseñas personales por título.

Seguimiento de episodios por temporada con indicador de progreso.

Historial de visionado agrupado por mes.

Caché de resultados TMDB en base de datos para minimizar peticiones externas.

Rol de administrador: gestión de usuarios y notificación de solicitudes de baja.

Solicitud de eliminación de cuenta por parte del usuario.

Despliegue en producción accesible públicamente desde cualquier dispositivo.

2. Elementos no vistos en clase

2.1. PHP 8.2

PHP (Hypertext Preprocessor) es un lenguaje de programación de propósito general orientado principalmente al desarrollo web del lado del servidor. Es de código abierto, interpretado y se ejecuta en el servidor, devolviendo el resultado al cliente (en este caso en formato JSON, al tratarse de una API REST).

La versión 8.2, utilizada en este proyecto, incorpora mejoras de rendimiento importantes respecto a versiones anteriores, como el compilador JIT (Just-In-Time), propiedades de solo lectura (readonly), enumeraciones nativas y tipos de unión e intersección. También introduce mejoras en el manejo de errores y en la seguridad por defecto.

Aspectos de PHP 8.2 utilizados en este proyecto:

Tipado estricto: declaración de tipos en parámetros y valores de retorno de funciones.

Atributos y clases anónimas para la integración con el framework CodeIgniter 4.

Extensiones pdo_pgsql y pgsql para la comunicación con la base de datos PostgreSQL.

Extensión JSON para la codificación y decodificación de datos entre la API y los clientes.

Funciones de hash (password_hash, password_verify) para el almacenamiento seguro de contraseñas.

2.2. CodeIgniter 4

CodeIgniter 4 es un framework PHP ligero de código abierto orientado al desarrollo de aplicaciones web y APIs REST. Sigue el patrón MVC (Modelo-Vista-Controlador) y ofrece herramientas integradas para enrutamiento, migraciones de base de datos, filtros de petición y cliente HTTP.

Elementos de CodeIgniter 4 utilizados en el proyecto:

Sistema de rutas para definir los endpoints de la API REST.

Migraciones para gestionar el esquema de la base de datos de forma versionada (15 migraciones).

Filtros (JWTAuthFilter, AdminFilter, CorsFilter) para autenticación y control de acceso.

CURLRequest para consumir la API de TMDB desde el servidor.

Query Builder y modelos para las operaciones con PostgreSQL.

3. Información sobre el proyecto

3.1. Arquitectura general

La aplicación sigue una arquitectura cliente-servidor en tres capas:

Capa de presentación (Flutter): la app móvil se comunica con el backend mediante peticiones HTTP/JSON. Gestiona el estado con Provider y almacena el token JWT localmente con SharedPreferences.

Capa de negocio y API (CodeIgniter 4): API REST que expone endpoints para autenticación, búsqueda, gestión de listas, historial y administración. Los filtros JWT validan cada petición. El servicio TMDBService actúa como intermediario con la API externa, aplicando caché.

Capa de datos (PostgreSQL): almacena los datos propios de los usuarios (listas, valoraciones, historial, tokens) y la caché de resultados de TMDB.

La API de TMDB es consultada por el backend (nunca directamente desde el móvil), lo que permite centralizar la caché y proteger la API key.

3.2. Diseño de la base de datos

Entidades y relaciones

La base de datos consta de 8 tablas distribuidas en dos grupos:

Datos de usuario: users, user_lists, list_items, refresh_tokens, user_episode_views, user_watch_history.

Caché de contenido externo: media_items, tmdb_cache.

Relaciones principales:

Un usuario (users) puede tener múltiples listas (user_lists). 1:N

Una lista (user_lists) puede contener múltiples ítems (list_items). 1:N

Cada ítem de lista (list_items) referencia un contenido multimedia (media_items). N:1

Un usuario puede tener múltiples tokens de refresco activos (refresh_tokens). 1:N

Un usuario puede haber visto múltiples episodios (user_episode_views). 1:N

Un usuario tiene un historial de visionado (user_watch_history). 1:N

Esquema conceptual modificado y paso a tablas

Tabla: users

Almacena los datos de los usuarios registrados.

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
email	VARCHAR(255)	UNIQUE, NOT NULL	Correo electrónico
password	VARCHAR(255)	NOT NULL	Hash bcrypt de la contraseña
name	VARCHAR(150)	NOT NULL	Nombre del usuario
avatar_url	VARCHAR(500)	NULL	URL de avatar (opcional)
role	VARCHAR(20)	DEFAULT 'user'	Rol: 'user' o 'admin'
deletion_requested_at	TIMESTAMP	NULL	Fecha de solicitud de baja
created_at	TIMESTAMPTZ	NULL	Fecha de registro
updated_at	TIMESTAMPTZ	NULL	Última modificación

Tabla: media_items

Caché local de películas y series obtenidas de TMDB.

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
tmdb_id	INT	NOT NULL	ID del título en TMDB
media_type	VARCHAR(10)	CHECK ('movie','tv')	Tipo: 'movie' o 'tv'
title	VARCHAR(500)	NOT NULL	Título
overview	TEXT	NULL	Sinopsis
poster_path	VARCHAR(255)	NULL	Ruta del póster en TMDB
backdrop_path	VARCHAR(255)	NULL	Ruta del backdrop
release_date	DATE	NULL	Fecha de estreno
vote_average	DECIMAL(4,2)	DEFAULT 0	Valoración media TMDB
genres	JSONB	DEFAULT '[]'	Array de géneros
raw_data	JSONB	DEFAULT '{}'	Respuesta completa de TMDB
cached_at	TIMESTAMPTZ	NULL	Fecha de última actualización
—	UNIQUE	(tmdb_id, media_type)	Sin duplicados por tipo

Tabla: user_lists

Listas personalizadas de cada usuario. Al registrarse se crean dos listas por defecto.

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
user_id	BIGINT	FK → users.id CASCADE	Usuario propietario
name	VARCHAR(150)	NOT NULL	Nombre de la lista
description	TEXT	NULL	Descripción opcional
is_public	BOOLEAN	DEFAULT FALSE	Visibilidad pública
is_default	BOOLEAN	DEFAULT FALSE	Lista creada automáticamente
media_type	VARCHAR(10)	NULL	Restricción de tipo ('movie','tv') o NULL
created_at / updated_at	TIMESTAMPTZ	NULL	Marcas temporales

Tabla: list_items

Relación entre una lista y un contenido multimedia, con datos propios del usuario.

Campo	Tipo	Restricciones	Descripción
-------	------	---------------	-------------

id	BIGINT	PK, AUTO	Identificador único
list_id	BIGINT	FK → user_lists.id CASCADE	Lista a la que pertenece
media_item_id	BIGINT	FK → media_items.id CASCADE	Contenido referenciado
user_note	TEXT	NULL	Reseña personal
user_rating	DECIMAL(3,1)	CHECK 0-10, NULL	Valoración del usuario
watched	BOOLEAN	DEFAULT FALSE	¿Marcado como visto?
added_at	TIMESTAMPTZ	NULL	Fecha de adición
—	UNIQUE	(list_id, media_item_id)	Sin duplicados en la misma lista

Tabla: tmdb_cache

Almacena respuestas completas de la API de TMDB con tiempo de expiración (TTL).

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
cache_key	VARCHAR(255)	UNIQUE, NOT NULL	Clave de caché única por petición
endpoint	VARCHAR(500)	NOT NULL	Endpoint de TMDB consultado
response	JSONB	NOT NULL	Respuesta completa de TMDB
expires_at	TIMESTAMPTZ	NOT NULL	Fecha de expiración de la caché
created_at	TIMESTAMPTZ	NULL	Fecha de creación

Tabla: refresh_tokens

Tokens de larga duración para renovar la sesión sin necesidad de hacer login de nuevo.

Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
user_id	BIGINT	FK → users.id CASCADE	Usuario propietario
token_hash	VARCHAR(255)	NOT NULL	Hash SHA-256 del token
expires_at	TIMESTAMPTZ	NOT NULL	Fecha de expiración
revoked	BOOLEAN	DEFAULT FALSE	¿Token revocado?
created_at	TIMESTAMPTZ	NULL	Fecha de creación

Tabla: user_episode_views

Registro de episodios vistos por cada usuario, con unicidad por episodio.

Campo	Tipo	Restricciones	Descripción
id	SERIAL	PK, AUTO	Identificador único
user_id	INTEGER	FK → users.id CASCADE	Usuario
tmdb_series_id	INTEGER	NOT NULL	ID de la serie en TMDB
season_number	SMALLINT	NOT NULL	Número de temporada
episode_number	SMALLINT	NOT NULL	Número de episodio
watched_at	TIMESTAMP	DEFAULT NOW()	Fecha de marcado
—	UNIQUE	(user_id, tmdb_series_id, season_number, episode_number)	

Tabla: user_watch_history

Historial cronológico de títulos marcados como vistos, agrupable por mes en la app.

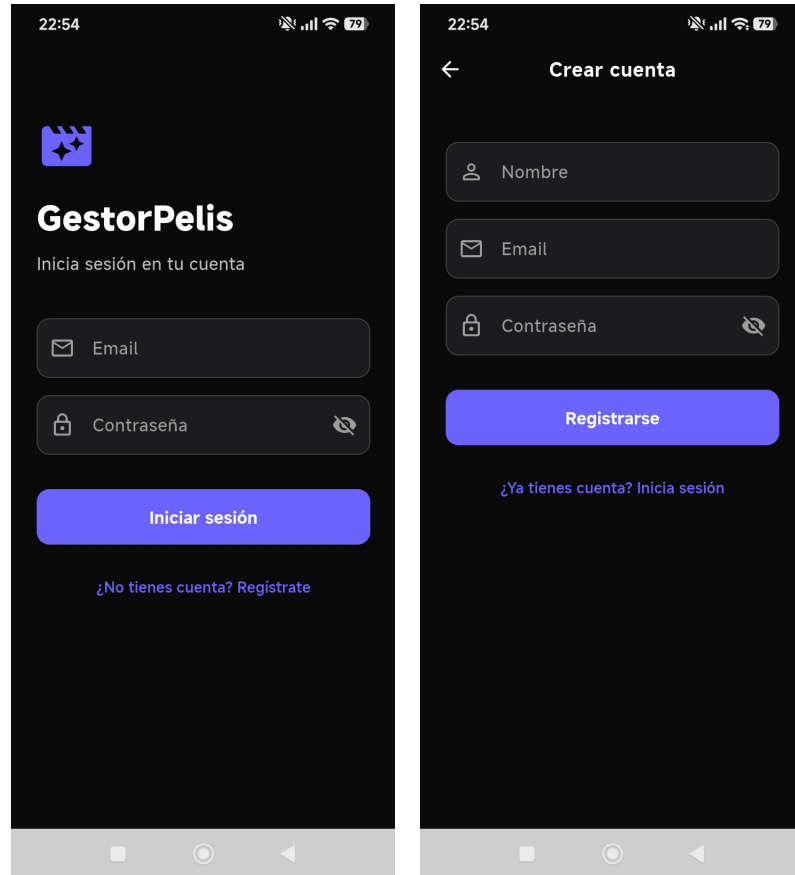
Campo	Tipo	Restricciones	Descripción
id	BIGINT	PK, AUTO	Identificador único
user_id	BIGINT	FK → users.id CASCADE	Usuario
tmdb_id	INTEGER	NOT NULL	ID del título en TMDB
media_type	VARCHAR(10)	NOT NULL	Tipo: 'movie' o 'tv'
title	VARCHAR(500)	NOT NULL	Título (desnormalizado)
poster_path	VARCHAR(500)	NULL	Ruta del póster
watched_on	DATE	NOT NULL	Fecha de visionado
created_at	TIMESTAMPTZ	NULL	Fecha de registro

3.3. Capturas de pantalla de la aplicación

A continuación se muestran las pantallas principales de la aplicación GestorPelis.

Inicio de sesión y registro

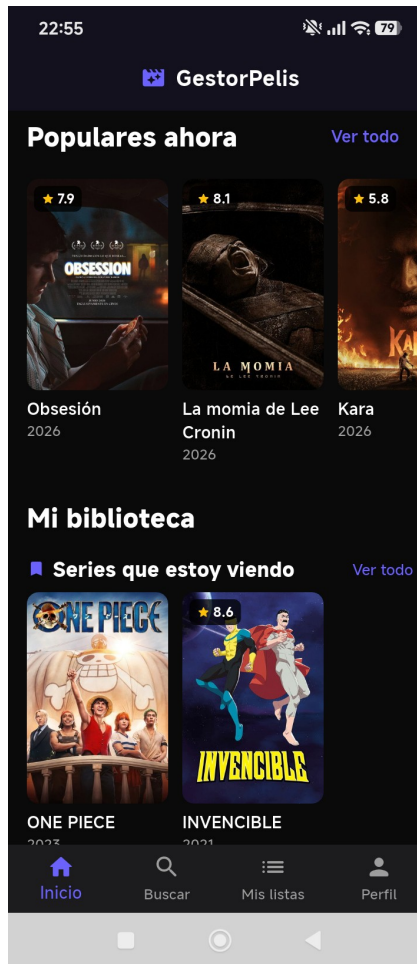
Al abrir la app el usuario ve la pantalla de inicio de sesión. Si no tiene cuenta, puede navegar a la pantalla de registro introduciendo nombre, correo y contraseña.



Pantalla de inicio de sesión (izq.) y registro (dcha.)

Pantalla principal

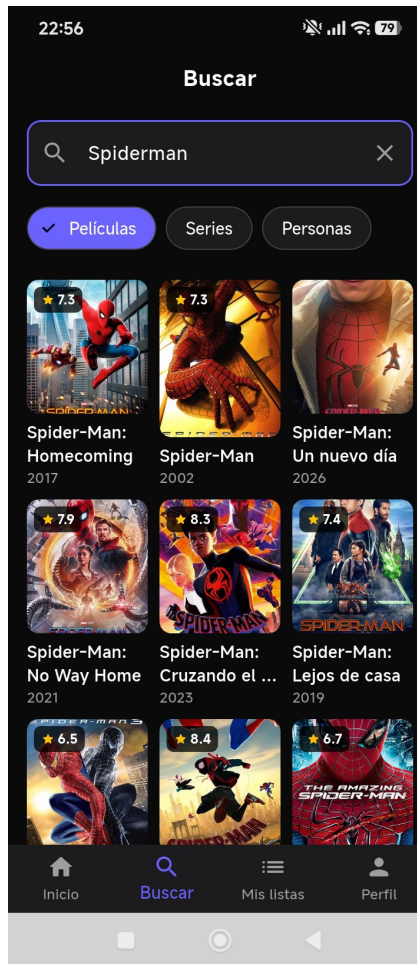
Tras iniciar sesión el usuario accede al inicio, que muestra un carrusel de contenido popular y, a continuación, el contenido de sus listas (biblioteca personal).



Pantalla principal con carruseles de contenido

Búsqueda

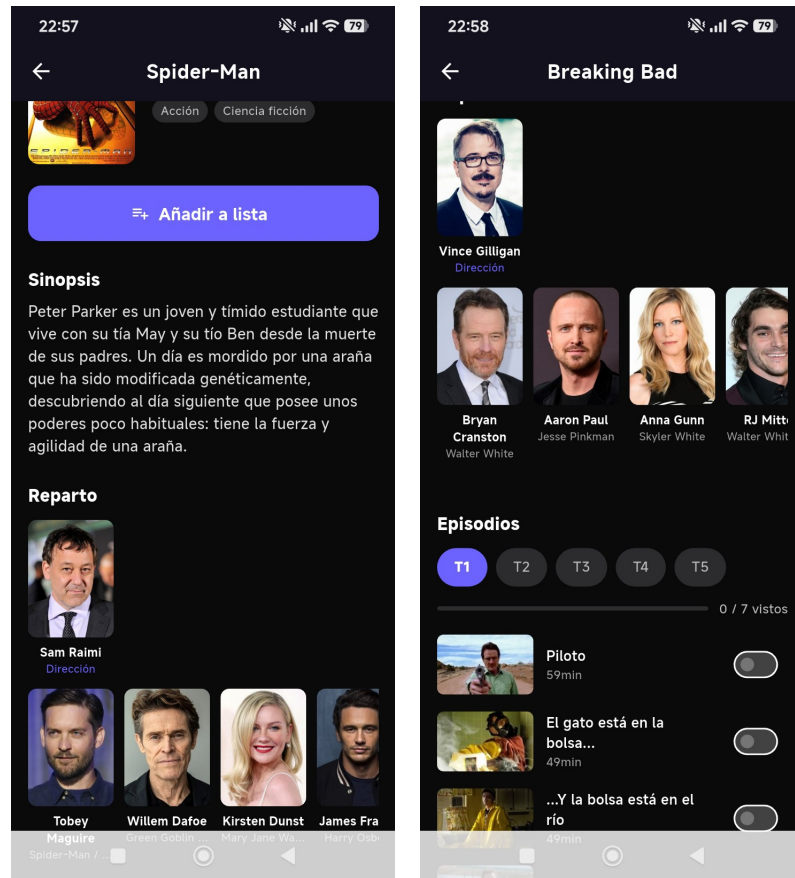
La pestaña Buscar permite localizar películas, series o personas por título. Los resultados se obtienen de la API de TMDB y se pueden filtrar por tipo.



Búsqueda de películas por título

Detalle de película y serie

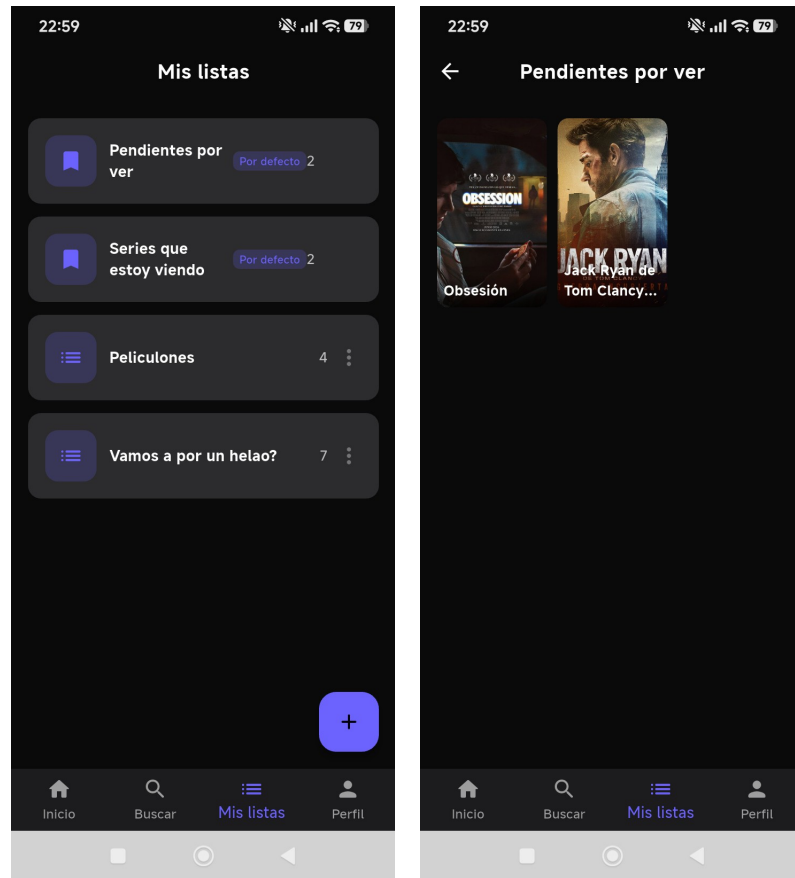
La pantalla de detalle muestra la sinopsis, géneros, reparto y la opción de añadir a una lista. Para las series se incluyen las temporadas con la lista de episodios y un toggle para marcar cada uno como visto.



Detalle de película (izq.) y serie con episodios (dcha.)

Listas de usuario

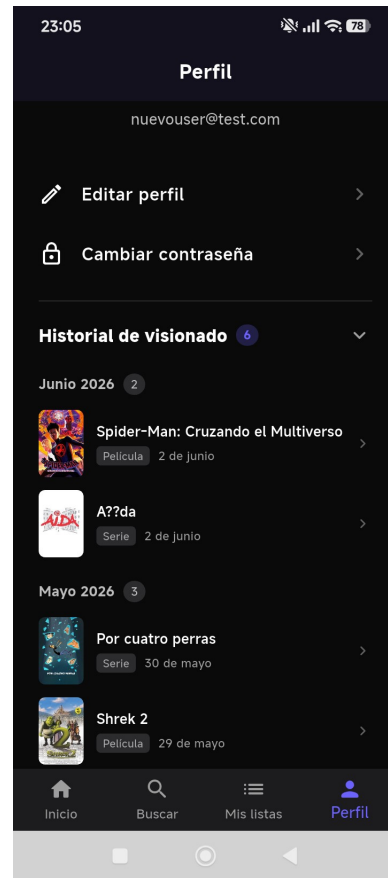
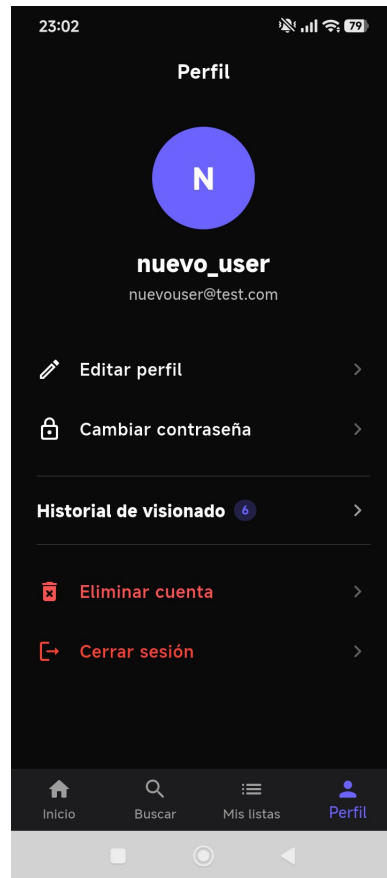
La sección Mis listas muestra todas las listas del usuario. Las dos listas por defecto (Pendientes por ver y Series que estoy viendo) aparecen siempre en primer lugar. Al entrar en una lista se ven sus títulos con la opción de deslizar para eliminar.



Listado de listas (izq.) y contenido de una lista (dcha.)

Perfil e historial de visionado

El perfil muestra los datos del usuario con opciones de edición y cambio de contraseña. El historial de visionado lista todos los títulos marcados como vistos, agrupados por mes.



Perfil de usuario (izq.) e historial de visionado desplegado (dcha.)

4. Parte práctica

4.1. Herramientas utilizadas

Software

Herramienta	Versión	Uso en el proyecto
PHP	8.2	Lenguaje del backend
CodeIgniter 4	4.5	Framework del backend (API REST)
PostgreSQL	16	Base de datos relacional
Docker Desktop	24.x	Contenedorización del entorno de desarrollo
Flutter / Dart	3.x	Desarrollo de la app móvil Android
Android SDK	API 26+	Compilación de la app Android
Composer	2.x	Gestor de dependencias PHP
Git	2.x	Control de versiones
Visual Studio Code	Reciente	Editor de código
Railway	—	Plataforma de despliegue en producción
TMDB API	v3	API externa de películas y series

Hardware

Componente	Descripción
PC de desarrollo	Windows 11 Pro, procesador x64, 32 GB RAM
Dispositivo de pruebas	Xiaomi Redmi 14C — Android 16 (pruebas de la app)

4.2. Instalación del entorno de desarrollo

Requisitos previos

Docker Desktop instalado y en ejecución.

Git instalado.

Flutter SDK instalado (para desarrollo de la app).

Backend (API REST)

Pasos para levantar el backend en local:

1. Clonar el repositorio: `git clone https://github.com/josesorianoe/tfg_gestorpelis.git`
2. Acceder a la carpeta del proyecto: `cd tfg_gestorpelis`
3. Copiar el archivo de entorno: `cp .env.example .env`
4. Editar `.env` y configurar: `TMDB_API_KEY` (obtenida en themoviedb.org) y `JWT_SECRET` (cadena aleatoria larga).
5. Levantar los contenedores Docker: `docker compose up -d`
6. Instalar dependencias PHP: `docker compose exec app composer install`

7. Ejecutar las migraciones: `docker compose exec app php spark migrate`
8. (Opcional) Cargar datos de prueba: `docker compose exec app php spark db:seed DatabaseSeeder`

La API quedará disponible en `http://localhost:8080`

Cómo obtener la API key de TMDB

1. Crear una cuenta gratuita en <https://www.themoviedb.org>
2. Ir a Ajustes de la cuenta → API.
3. Solicitar una API key de tipo Developer.
4. Copiar la clave (v3 auth) y pegarla en `TMDB_API_KEY` del archivo `.env`.

Aplicación Flutter

1. Acceder a la carpeta de la app: `cd mobile`
2. Instalar dependencias: `flutter pub get`
3. Conectar un dispositivo Android con depuración USB activada.
4. Ejecutar en modo desarrollo: `flutter run`
5. Para compilar el APK de producción: `flutter build apk --release`

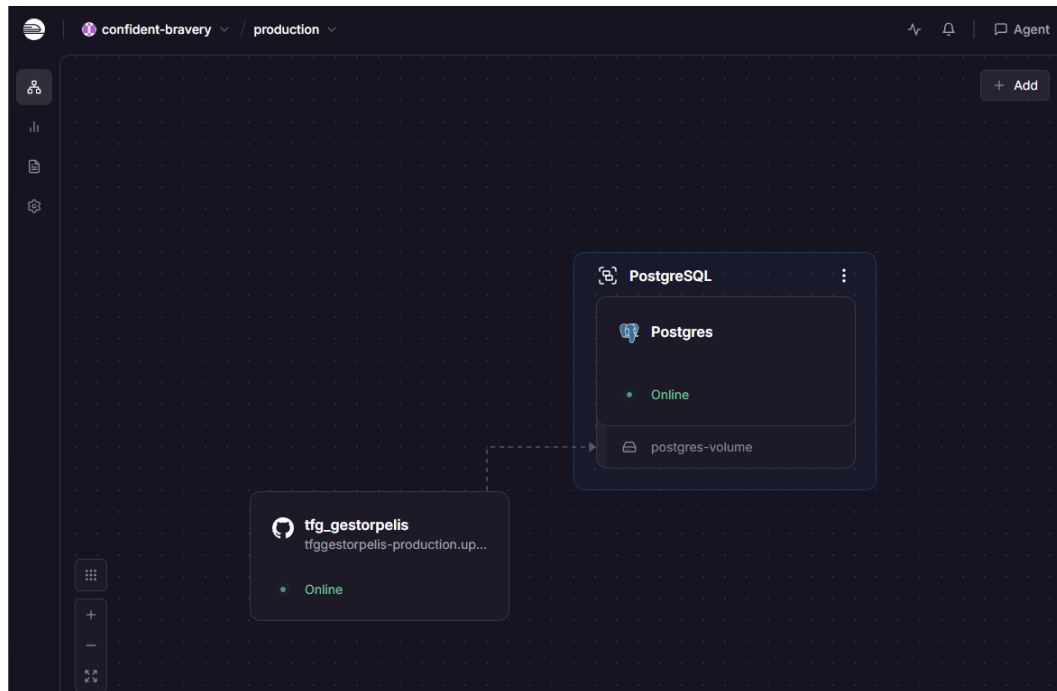
4.3. Despliegue en producción (Railway)

El backend está desplegado en Railway, una plataforma PaaS que permite desplegar aplicaciones directamente desde GitHub usando Docker.

Proceso de despliegue

1. Crear una cuenta en railway.app y crear un nuevo proyecto.
2. Conectar el repositorio de GitHub ([josesorianoe/tfg_gestorpelis](https://github.com/josesorianoe/tfg_gestorpelis)).
3. Añadir el plugin de PostgreSQL al proyecto.
4. Configurar las variables de entorno: `DB_HOST`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `JWT_SECRET`, `TMDB_API_KEY`, `APP_BASE_URL`.
5. Railway detecta automáticamente el archivo `Dockerfile.prod` y construye la imagen.
6. Al completarse el build, el backend queda accesible en la URL pública asignada.
7. Las migraciones se ejecutan automáticamente en cada despliegue mediante el script `start.sh`.

URL del backend en producción: <https://tfggestorpelis-production.up.railway.app>



Panel de Railway: servicio backend y PostgreSQL en estado Online

5. Bibliografía

- [1] PHP 8 y MYSQL: El Curso Completo, Práctico y Desde Cero !
https://www.udemy.com/share/101XoI3@cu0rsMyFXLy0-0oC1P_oLOUdaAzpyxyLeh5L2z1kJW-axK-vaqrSq8qEAYWSm5sTMw==/
- [2] [Curso] Codeigniter 4 de 0 a 100. https://www.youtube.com/watch?v=iN4QirLvEHs&list=PLCTD_CpMeEKTFN5TDeOP-wP_hU0_9VoWg
- [3] Documentación oficial de CodeIgniter 4. https://codeigniter.com/user_guide/
- [4] Documentación oficial de Flutter. <https://docs.flutter.dev/>
- [5] Documentación de la API de TMDb. <https://developer.themoviedb.org/docs>
- [6] Documentación oficial de Docker. <https://docs.docker.com/>
- [7] Documentación de Railway. <https://docs.railway.app/>
- [8] Curso de Base de Datos PostgreSQL. <https://www.youtube.com/watch?v=gEjcMrk3E-Q&list=PLgqdACsQ8US2DCzQVrdZDZCTtTFHMY0as>
- [9] Documentación de PostgreSQL 16. <https://www.postgresql.org/docs/16/>
- [10] Paquete Provider para Flutter. <https://pub.dev/packages/provider>
- [11] Paquete go_router para Flutter. https://pub.dev/packages/go_router
- [12] Claude.